

第12章 注解与元编程

建议学时:4-6 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 理解注解的本质:编译器与框架可读的结构化元数据
- 掌握注解定义语法:@interface、元素类型、默认值
- 掌握四个核心元注解:Retention / Target / Documented / Inherited
- 掌握运行时注解处理:反射读取与执行校验逻辑
- 了解编译期注解处理(APT)与内置注解
- 掌握注解驱动的工程模式:校验器、配置绑定

💡 从 C 到 Java:会执行的"注释"

C 程序员对"注释"的心智:给人看的,编译器忽略。Java 的 **注解(Annotation)** 是给程序看的"注释"——编译器、框架、工具可以读取并据此行动的结构化元数据。第11章"借用"过它:字段上的 @Column 注解,让一个不认识你的框架知道"这个字段导出叫什么名字"。

注解 = 声明式编程。你声明"这个字段不能为 null",而不是写"if (x == null) throw ...";框架在运行时读注解,替你执行校验。好处:业务代码只写"是什么",通用逻辑(校验、序列化、注入)在框架里写一次,对所有带注解的类生效。这就是"配置与代码分离"的终极形态——配置就在代码里,但以元数据的形式。

C 里最接近的东西是什么?没有直接对应物。最像的是:代码生成器读取的特殊标记注释 + 函数指针表。X-Macro、__attribute__((...)) 有一点味道,但注解是类型安全、可携带参数、可被反射查询的正式语言设施。

Retention 决定注解的命运。SOURCE(编译完就扔,如 @Override)、CLASS(进字节码,默认)、RUNTIME(进字节码且反射可读)。想要"运行时读取执行"的注解,必须标 RUNTIME——这是新手第一坑:定义得挺好,反射却读不到。

学习顺序:先懂"注解是什么"与内置注解(\$12.1),定义语法(\$12.2),元注解(\$12.3),运行时处理(\$12.4),编译期处理(\$12.5),工程模式(\$12.6)。

📦 前置知识自查

- 第11章:反射读字段与注解(本章直接续写)
- 第6章:封装与字段;第10章:泛型(注解元素与泛型约束)
- 能识别 @Override/@Deprecated 等见过的注解

🚀 本章学习路线

1. **第一阶段(约 1 小时):**\$12.1-12.2 定义与元注解,写出自己的第一个 @interface
2. **第二阶段(约 1.5 小时):**\$12.3-12.4 Retention/Target 与运行时处理,实现字段校验器
3. **第三阶段(约 1.5 小时):**\$12.5-12.6 编译期处理与工程模式,完成章末实验
4. **验收标准:**能定义带默认值注解;能解释四个元注解;能写出运行时校验器

本章知识导图

- §12.1 注解的本质:元数据 / 内置注解
- §12.2 定义注解:@interface / 元素 / 默认值
- §12.3 元注解:Retention / Target / Documented / Inherited
- §12.4 运行时处理:反射读取与执行
- §12.5 编译期处理:APT 与内置注解
- §12.6 注解工程模式:校验器 / 配置绑定

§12.1 注解的本质与内置注解

12.1.1 注解 vs 注释 vs C 的替代品

对比项	C 注释	Java 注解
读者	人	编译器、框架、工具(人也读)
是否进产物	编译后消失	按 Retention:可进字节码、可反射
能否携带数据	自由文本(无法结构化读取)	结构化元素(类型检查的键值对)
能否触发行为	不能	能:编译器检查、处理器生成代码、框架执行逻辑
C 的近似物	/* 注释 */、X-Macro 宏	(无直接对应;__attribute__ 略近)

12.1.2 三个最常用的内置注解

注解	作用	谁消费它
@Override	标记"这是重写父类方法"	编译器:签名写错立即报错(第6章)
@Deprecated	标记"别用了,有替代"	编译器:使用时给警告;IDE 划线
@SuppressWarnings("...")	压制特定警告(如 unchecked)	编译器

示例 12-1 内置注解实战

```
import java.util.List;

public class BuiltinAnnotations {
    // @Deprecated:标记旧 API,调用处会警告
    @Deprecated
    static void oldMethod() {
        System.out.println("旧方法,请用 newMethod");
    }

    static void newMethod() {
        System.out.println("新方法");
    }

    // @SuppressWarnings:压掉"裸类型"警告(旧代码迁移时的过渡手段)
    @SuppressWarnings({"unchecked", "rawtypes"})
    static List rawList() {
        return new java.util.ArrayList(); // 裸类型:正常会警告
    }

    public static void main(String[] args) {
        newMethod();
        oldMethod(); // IDE 会提示"已弃用"
        System.out.println(rawList().size());
    }
}
```

★ 重点:注解自己不做任何事

注解本身是"标签",行为全部来自消费者: `@Override` 没写对,编译器检查; `@Deprecated` 没处理,警告;自定义注解没人读,就是装饰。所以学注解 = 学"定义" + 学"消费"。消费有两类:编译器(编译期处理,§12.5)与反射/框架(运行时处理,§12.4)。

§12.2 定义注解:@interface

12.2.1 语法与元素规则

定义用 `@interface` 关键字;里面声明"元素"(像方法声明,没有方法体):元素类型只允许 8 种基本类型、String、Class、枚举、注解、以及它们的数组;可以有默认值(default)。使用时不写有默认值的元素。

示例 12-2 定义自己的注解

```

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

// 字段校验注解:标记"这个字段不能为空"
@Retention(RetentionPolicy.RUNTIME)    // 运行时保留(反射可读)
@Target(ElementType.FIELD)              // 只能用在字段上
public @interface NotNull {
    // 元素:使用时可省略(有默认值)
    String message() default "字段不能为 null";

    // 元素:必须显式提供(无默认值)
    boolean checkEmpty();                // 空字符串也算非法
}

// 用法:把校验"声明"在字段上
class User {
    @NotNull(checkEmpty = true)
    private String name;

    @NotNull(checkEmpty = false, message = "邮箱可选,但不能为 null")
    private String email;
}

```

⚠ 易错点 1:元素类型与命名限制

元素类型有限制:基本类型/String/Class/枚举/注解/它们的数组——想存"任意对象"不行,想存 List 不行。命名惯例:只有一个元素时叫 value,使用时可省略 = 写成 @Column("stu_id") (第11章);多个元素时必须写 @NotNull(checkEmpty = true)。注解本身也是类型,可以嵌套(注解里放注解),框架里常见。

§12.3 元注解:注解的注解

元注解	作用	生产要点
@Retention	注解活多久	SOURCE 编译即弃;CLASS 进字节码;RUNTIME 反射可读(自定义运行逻辑必用)
@Target	注解能标在哪	TYPE 类/接口 / FIELD 字段 / METHOD 方法 / PARAMETER 参数 / CONSTRUCTOR 构造器 / ANNOTATION_TYPE 注解 / PACKAGE 包;不写 = 到处都能标(不推荐)
@Documented	进 javadoc	公开 API 注解建议加
@Inherited	子类继承父类的注解	只对类上的注解有效;字段/方法注解不继承

示例 12-3 元注解组合拳

```
import java.lang.annotation.Documented;
import java.lang.annotation.ElementType;
import java.lang.annotation.Inherited;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

// 类级别的"权限"注解:可继承、进 javadoc
@Documented
@Inherited
@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.TYPE, ElementType.METHOD})
public @interface RequiresAuth {
    String role() default "user";    // 需要的角色

    boolean audit() default false;   // 是否记录审计日志
}

// 用法:类上标一次,子类自动继承
@RequiresAuth(role = "admin", audit = true)
class AdminApi { }

class SubAdminApi extends AdminApi { }    // 自动继承 @RequiresAuth
```

⚠ 易错点 2:Retention 没设成 RUNTIME,反射读不到

默认 Retention 是 CLASS:注解进字节码,但 JVM 不保留给反射。getAnnotation 返回 null,代码不报错,就是读不到——最隐蔽的注解 bug。自检顺序:① RetentionPolicy.RUNTIME 写了没;② Target 包含你要标的元素类型没;③ 反射用 getDeclaredAnnotation 而不是 getAnnotation(后者对继承的处理不同)。

示例 12-4 读取类级注解(含 @Inherited 效果)

```
import java.lang.reflect.Method;

public class ReadClassAnno {
    // 读取类的权限注解:getAnnotation 对类级注解会沿继承链查找
    static String roleOf(Class<?> type) {
        RequiresAuth ra = type.getAnnotation(RequiresAuth.class);
        return ra == null ? "(无权限要求)" : ra.role();
    }

    // 读取方法上的注解(不继承:重写后是新方法)
    static boolean auditOf(Class<?> type, String methodName) throws Exception {
        Method m = type.getMethod(methodName);
        RequiresAuth ra = m.getAnnotation(RequiresAuth.class);
        return ra != null && ra.audit();
    }

    public static void main(String[] args) throws Exception {
        System.out.println(roleOf(AdminApi.class));    // admin
        System.out.println(roleOf(SubAdminApi.class)); // admin:继承生效
    }
}

// 注:RequiresAuth 定义见示例 12-3,此处省略
```

§12.4 运行时处理:注解校验器

12.4.1 处理流程:扫描 → 读注解 → 执行逻辑

运行时处理的标准三步(第11章已预热):① getDeclaredFields 拿到字段;② 字段.getAnnotation(X.class) 读注解;③ 按注解参数执行通用逻辑。下面实现一个完整的校验器——生产校验框架(Hibernate Validator)就是它的工程化版本。

示例 12-4 运行时注解校验器

```

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.reflect.Field;
import java.util.ArrayList;
import java.util.List;

public class Validator {
    // 注解 1:不能为 null/空串
    @Retention(RetentionPolicy.RUNTIME)
    @Target(ElementType.FIELD)
    public @interface NotEmpty {
        String message() default "字段不能为空";
    }

    // 注解 2:数值范围
    @Retention(RetentionPolicy.RUNTIME)
    @Target(ElementType.FIELD)
    public @interface Range {
        int min() default 0;

        int max() default Integer.MAX_VALUE;

        String message() default "数值越界";
    }

    // 通用校验:不认识 Order 类也能校验它
    static List<String> validate(Object obj) throws IllegalAccessException {
        List<String> errors = new ArrayList<>();
        for (Field f : obj.getClass().getDeclaredFields()) {
            f.setAccessible(true);
            Object value = f.get(obj);

            NotEmpty ne = f.getAnnotation(NotEmpty.class);
            if (ne != null && (value == null
                || (value instanceof String s && s.isEmpty())))) {
                errors.add(f.getName() + ": " + ne.message());
            }

            Range r = f.getAnnotation(Range.class);
            if (r != null && value instanceof Number n
                && (n.intValue() < r.min() || n.intValue() > r.max())) {
                errors.add(f.getName() + ": " + r.message()
                    + "(范围 " + r.min() + "~" + r.max() + ")");
            }
        }
        return errors;
    }

    // 业务类:声明校验规则
    static class Order {
        @NotEmpty
        private String orderId;

        @NotEmpty(message = "收件人必须填写")
        private String receiver;

        @Range(min = 1, max = 9999, message = "数量不合法")
        private int quantity;

        // 构造器(演示用)
        Order(String orderId, String receiver, int quantity) {
            this.orderId = orderId;
            this.receiver = receiver;
            this.quantity = quantity;
        }
    }
}

```

```

public static void main(String[] args) throws Exception {
    Order good = new Order("A1001", "张三", 3);
    System.out.println("合法订单校验: " + validate(good));    // []

    Order bad = new Order("", " ", 0);
    System.out.println("非法订单校验: " + validate(bad));
    // [orderId: 字段不能为空, receiver: 收件人必须填写,
    //   quantity: 数量不合法(范围 1~9999)]
}
}

```

★ 重点:声明式校验的三层收益

① 规则与字段在一起:读 Order 类源码,一眼看到每个字段的约束,不用翻校验方法;② 校验逻辑只写一次:新类加注解即获校验能力,validate 一行不改;③ 规则可组合:一个字段可以同时挂 @NotEmpty + @Range + 自定义注解,框架按顺序执行。这就是"声明式"vs"命令式"的差别:你声明"是什么",框架负责"怎么做"。

§12.5 编译期处理与内置注解

12.5.1 两种消费时机

消费时机	工具	典型应用	是否进运行时
编译期	注解处理器 (APT,javac - processor)	Lombok 生成 getter、 DataBinding 生成代码、 Google AutoService	处理完即消失(或生成新源码)
运行时	反射(getAnnotation + 逻辑)	校验器(§12.4)、Spring 注 入、MyBatis 映射	必须 RetentionPolicy.RUNTIME

示例 12-5 编译期处理的最小形态(概念演示)

```

// 编译器看到 @Override 时做的"处理",是内置注解处理器的范例:
// 1. 扫描目标类的所有方法
// 2. 对每个标注 @Override 的方法,检查父类/接口是否有同签名方法
// 3. 没有 → 编译错误 "method does not override a method from its superclass"

// 同理 @Deprecated:
// 1. 扫描所有方法调用
// 2. 若目标方法带 @Deprecated 且调用点未同时标注
// 3. 生成警告,IDE 划线

// 自研 APT 的骨架(javax.annotation.processing,JDK 老 API):
// @SupportedAnnotationTypes("com.example.MyAnno")
// public class MyProcessor extends AbstractProcessor {
//     @Override
//     public boolean process(Set<? extends TypeElement> annotations,
//                             RoundEnvironment roundEnv) {
//         // 扫描带注解的元素,生成新 .java 文件或报错
//         return true;
//     }
// }

```

🚀 工程建议:什么时候用编译期,什么时候用运行时

需要"生成代码"或"编译期报错"→ APT(Lombok、Builder 生成);需要"读取对象运行时状态再决策"→ 反射(校验、注入、序列化)。APT 生成代码性能最好(没有反射开销),但开发体验重(要写处理器、处理增量编译);反射方案简单直接,代价是性能与可读性。多数业务项目的自定义注解走运行时,框架级工具才上 APT。

§12.6 注解工程模式:配置绑定

12.6.1 把 Properties 绑定到带注解的类

生产里的"配置类"模式:定义配置类,字段打上注解声明"从哪个配置键读取、默认值是什么",框架一行代码把 Properties 绑定成类型安全的配置对象——Spring 的 `@ConfigurationProperties` 就是这个模式的巅峰。

示例 12-6 注解驱动的配置绑定器


```

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.reflect.Field;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Properties;

public class ConfigBinder {
    // 配置键注解
    @Retention(RetentionPolicy.RUNTIME)
    @Target(ElementType.FIELD)
    public @interface Prop {
        String key();

        String def() default "";    // 默认值(字符串形态)
    }

    // 通用绑定:把 Properties 灌进任意"打了注解的类"
    static <T> T bind(Class<T> type, Properties props) throws Exception {
        T cfg = type.getDeclaredConstructor().newInstance();
        for (Field f : type.getDeclaredFields()) {
            Prop p = f.getAnnotation(Prop.class);
            if (p == null) {
                continue;
            }
            String raw = props.getProperty(p.key(), p.def());
            f.setAccessible(true);
            Class<?> ft = f.getType();
            if (ft == int.class || ft == Integer.class) {
                f.setInt(cfg, Integer.parseInt(raw.trim()));
            } else if (ft == boolean.class || ft == Boolean.class) {
                f.setBoolean(cfg, Boolean.parseBoolean(raw.trim()));
            } else {
                f.set(cfg, raw);
            }
        }
        return cfg;
    }

    // 配置类:声明"我要什么配置"
    static class ServerConfig {
        @Prop(key = "server.port", def = "8080")
        private int port;

        @Prop(key = "server.name", def = "unnamed")
        private String name;

        @Prop(key = "server.debug", def = "false")
        private boolean debug;

        @Override
        public String toString() {
            return "ServerConfig{port=" + port + ", name='" + name
                + "', debug=" + debug + "}";
        }
    }

    public static void main(String[] args) throws Exception {
        Properties props = new Properties();
        Path f = Path.of("server.properties");
        if (Files.exists(f)) {
            try (var in = Files.newBufferedReader(f, StandardCharsets.UTF_8)) {
                props.load(in);
            }
        }
    }
}

```

```
// 模拟配置文件内容:
props.setProperty("server.port", "9001");
props.setProperty("server.name", "主站");

ServerConfig cfg = bind(ServerConfig.class, props);
System.out.println(cfg);
// ServerConfig{port=9001, name='主站', debug=false}
// port 与 name 来自配置,debug 用默认值:绑定器正确合并
}
}
```

⚠ 易错点 3:注解默认值 vs 配置默认值,别混

两套默认值要分清: 注解元素的 default(如 def="8080")是"代码里写死的缺省";Properties 里没有该键时用它。真实配置有值则覆盖两者。生产还有第三层:环境变量覆盖配置(第7章)——绑定器的优先级链:环境变量 > 配置文件 > 注解默认值。设计配置系统时把这条链写清楚,是大型项目的必修课。

本章速查表

主题	要点
注解本质	结构化元数据;标签本身不做事,行为来自消费者(编译器/框架)
定义语法	public @interface X { 元素(); 元素() default 值; }
元素类型	基本类型/String/Class/枚举/注解/数组;不能存任意对象或 List
@Retention	SOURCE 编译即弃 / CLASS 进字节码 / RUNTIME 反射可读(自定义运行逻辑必选)
@Target	TYPE/FIELD/METHOD/PARAMETER/CONSTRUCTOR/ANNOTATION_TYPE/PACKAGE;多值用花括号
@Inherited	类注解可被子类继承;字段/方法注解不继承
读取	field.getAnnotation(X.class) → null 表示没有;Retention 不对就永远 null
运行时处理	getDeclaredFields → getAnnotation → 执行逻辑;校验器/注入/序列化
编译期处理	APT 注解处理器:生成源码/编译报错;Lombok 的原理
内置注解	@Override(编译检查)/ @Deprecated(警告)/ @SuppressWarnings(压警告)
工程模式	声明式校验、配置绑定;优先级链:环境变量 > 配置文件 > 注解默认值

最小必会清单

- 能解释"注解是给程序看的注释"与三种消费方式
- 能写出带默认值与必填元素的注解定义
- 能说出四个元注解的作用与各自的生产要点
- 能解释"Retention 不是 RUNTIME 时反射读不到"的排查顺序
- 能写出字段校验器的完整代码(扫描-读取-执行)
- 能说出 APT 与运行时处理的适用场景
- 能写出配置绑定器,并说出默认值优先级链

自测题

Q 1. 你定义了一个注解,反射却读不到。列出按顺序排查的三步。

✓ 参考答案

① Retention 是否为 RUNTIME(默认 CLASS 只进字节码,反射不可见);② Target 是否包含你要标注的元素类型(标到类上却 @Target(FIELD) 会编译错误,但标错"目标"是另一个坑:用了 @Target 未列出的位置会编译失败,说明 Target 限制了使用处);③ 读取 API 是否正确(getDeclaredAnnotation 管本类声明,getAnnotation 涉及继承规则)。

Q 2. 注解元素为什么不能是 List<String>?想要"多个值"怎么办?

✓ 参考答案

注解元素必须能在编译期常量折叠并写入字节码的"常量表",List 是运行时对象,装不进常量表。多值用数组: `String[] roles() default {"user"}`,使用 `@RequiresAuth(roles = {"admin", "ops"})`。枚举与注解也可以作元素类型,组合出嵌套结构。

Q 3. @Inherited 对方法上的注解生效吗?为什么?

✓ 参考答案

不生效。@Inherited 只对类级注解(标注在 TYPE 上的)生效:子类继承父类的类注解。方法被"重写"而非"继承"——重写后方法本体是新的,注解跟着方法声明走,所以方法注解不会自动传给重写版。需要"方法级继承"得自己处理:反射时沿父类找同名方法读注解。

Q 4. 校验器里,一个字段同时标了 @NotEmpty 与 @Range,执行顺序怎么保证?有什么坑?

✓ 参考答案

getDeclaredFields 的顺序不保证——两个注解的执行顺序在反射层面不可控(而且数组里多字段整体顺序也不保证)。坑:先执行 Range 时 value 为 null,Number 判断会跳过;先执行 NotEmpty 则正常。生产框架(Hibernate Validator)有明确的组与顺序机制;自研校验器要在文档里声明"顺序不保证",或按注解的显式 order 值排序。

Q 5. Lombok 的 @Getter 在编译期生成 getter——为什么不能靠运行时反射实现同样的效果?

✓ 参考答案

运行时反射可以"模拟读取",但调用方写的是 `user.getName()`——编译期就需要这个方法存在,反射无法让编译通过。Lombok 在编译期往字节码里真正生成 getName 方法,所以源码里不写、编译产物里有。这就是 APT 的典型价值:生成"编译期就要存在"的代码;运行时反射解决"编译期不知道类型"的问题。两者互补,选型看需求的时机。

Q 6. 配置绑定器里,int 字段 parse 失败(配置写成了 "abc"),程序会怎样?生产应该怎么做?

✓ 参考答案

NumberFormatException(RuntimeException)直接抛出,启动失败——这其实是"好的失败":配置错误必须在启动时暴露,而不是运行到一半才炸。生产增强:① 错误消息带键名("server.port 配置无法解析: abc");② 汇总所有字段的解析错误一次性报告(别修一个重启一次);③ 支持类型转换器注册表(第11章),把 String→LocalDate 等也纳管。

章末实验题:表单校验器 + 配置中心

实验 12:注解驱动的表单校验与配置绑定

实验目标:编写两部分程序:① 完整字段校验器(支持 @NotNull/@Range/@Regex 与链式错误汇总);② 配置绑定器(支持环境变量覆盖)。综合运用本章全部知识点。

实验要求:

- 任务 1:定义三个注解 @NotNull、@Range、@Regex(正则校验),均带 message 默认值,Retention 为 RUNTIME(覆盖 §12.2-12.3 定义与元注解)
- 任务 2:校验器 validate(Object):扫描全部字段(含父类),收集所有错误到 List,不抛异常(覆盖 §12.4 运行时处理)
- 任务 3:支持继承:父类字段也校验(覆盖 §12.4 继承链,第11章技巧)
- 任务 4:定义表单类 RegisterForm(用户名/年龄/邮箱),打上三个注解,演示合法与非法两组数据(覆盖 §12.4 业务)
- 任务 5:配置绑定器 bind(Class<T>, Properties):支持 int/boolean/String,缺失时用注解默认值(覆盖 §12.6 配置绑定)
- 任务 6:绑定器支持"环境变量覆盖":System.getenv("APP_PORT") 存在且非空时优先(覆盖 §12.6 优先级链)
- 任务 7:main 里演示:非法表单的错误清单、合法表单通过、配置绑定结果(覆盖全部)

实验环境:JDK 17+。

第一步 定义注解(不写处理逻辑),再写 validate 的骨架:遍历字段打印注解信息

第二步 任务 3 复用第11章的 while (c != null) 继承链遍历

第三步 任务 6 的环境变量:Windows 下先 set APP_PORT=9000 再运行(或代码里给默认路径)

参考答案要点

```

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.reflect.Field;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;
import java.util.Properties;
import java.util.regex.Pattern;

/** 注解驱动表单校验器 + 配置绑定 — 第12章综合实验参考答案
 * 覆盖:注解定义、元注解、运行时处理、继承链、校验执行、配置绑定、优先级链
 */
public class AnnotationLab {

    // ---- 任务 1:三个校验注解 ----
    @Retention(RetentionPolicy.RUNTIME)
    @Target(ElementType.FIELD)
    public @interface NotNull {
        String message() default "不能为空";
    }

    @Retention(RetentionPolicy.RUNTIME)
    @Target(ElementType.FIELD)
    public @interface Range {
        int min() default Integer.MIN_VALUE;

        int max() default Integer.MAX_VALUE;

        String message() default "数值越界";
    }

    @Retention(RetentionPolicy.RUNTIME)
    @Target(ElementType.FIELD)
    public @interface Regex {
        String pattern();

        String message() default "格式不合法";
    }

    // ---- 任务 2+3:校验器(含继承链)----
    static List<String> validate(Object obj) throws IllegalAccessException {
        List<String> errors = new ArrayList<>();
        for (Class<?> c = obj.getClass(); c != null; c = c.getSuperclass()) {
            for (Field f : c.getDeclaredFields()) {
                f.setAccessible(true);
                Object value = f.get(obj);
                String where = c.getSimpleName() + "." + f.getName();

                NotNull nn = f.getAnnotation(NotNull.class);
                if (nn != null && value == null) {
                    errors.add(where + " " + nn.message());
                }

                Range rg = f.getAnnotation(Range.class);
                if (rg != null && value instanceof Number num
                    && (num.intValue() < rg.min() || num.intValue() > rg.max())) {
                    errors.add(where + " " + rg.message() + "[" + rg.min()
                        + "," + rg.max() + "]");
                }

                Regex rx = f.getAnnotation(Regex.class);
                if (rx != null && value != null
                    && !Pattern.matches(rx.pattern(), String.valueOf(value))) {
                    errors.add(where + " " + rx.message());
                }
            }
        }
    }
}

```

```

    }
    }
    return errors;
}

// ---- 任务 4:表单类 ----
static class RegisterForm {
    @NotNull(message = "用户名必须填写")
    @Regex(pattern = "[a-zA-Z0-9_]{4,16}", message = "用户名需 4-16 位字母数字")
    private String username;

    @NotNull
    @Range(min = 18, max = 60, message = "年龄需在 18-60")
    private Integer age;

    @NotNull(message = "邮箱必须填写")
    @Regex(pattern = "^[^@\\s]+@[^@\\s]+\\.^[^@\\s]+$", message = "邮箱格式不正确")
    private String email;
}

// ---- 任务 5+6:配置绑定器(含环境变量覆盖)----
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface Prop {
    String key();

    String def() default "";

    String env() default "";    // 可选的环境变量名(优先级最高)
}

static class AppConfig {
    @Prop(key = "app.port", def = "8080", env = "APP_PORT")
    private int port;

    @Prop(key = "app.name", def = "未命名应用", env = "APP_NAME")
    private String name;

    @Prop(key = "app.debug", def = "false", env = "APP_DEBUG")
    private boolean debug;
}

static <T> T bind(Class<T> type, Properties props) throws Exception {
    T cfg = type.getDeclaredConstructor().newInstance();
    for (Field f : type.getDeclaredFields()) {
        Prop p = f.getAnnotation(Prop.class);
        if (p == null) continue;
        String raw = null;
        // 优先级:环境变量 > 配置文件 > 注解默认值
        if (!p.env().isEmpty()) {
            String env = System.getenv(p.env());
            if (env != null && !env.isBlank()) {
                raw = env;
            }
        }
        if (raw == null) {
            raw = props.getProperty(p.key(), p.def());
        }
        f.setAccessible(true);
        Class<?> ft = f.getType();
        if (ft == int.class || ft == Integer.class) {
            f.setInt(cfg, Integer.parseInt(raw.trim()));
        } else if (ft == boolean.class || ft == Boolean.class) {
            f.setBoolean(cfg, Boolean.parseBoolean(raw.trim()));
        } else {
            f.set(cfg, raw);
        }
    }
}

```

```

    return cfg;
}

public static void main(String[] args) throws Exception {
    System.out.println("== 任务 7a:表单校验 ==");
    RegisterForm bad = new RegisterForm();
    bad.username = "a"; // 太短
    bad.age = 17; // 太小
    bad.email = "not-an-email"; // 格式错
    System.out.println("非法表单错误清单:");
    validate(bad).forEach(e -> System.out.println(" - " + e));

    RegisterForm good = new RegisterForm();
    good.username = "zhangsan";
    good.age = 25;
    good.email = "zs@example.com";
    System.out.println("合法表单错误数: " + validate(good).size()); // 0

    System.out.println("== 任务 7b:配置绑定 ==");
    Properties props = new Properties();
    Path f = Path.of("app.properties");
    if (Files.exists(f)) {
        try (var in = Files.newBufferedReader(f, StandardCharsets.UTF_8)) {
            props.load(in);
        }
    }
    props.setProperty("app.name", "教务管理"); // 配置文件提供 name
    // 若设置了环境变量 APP_PORT,它会覆盖 props 里的值
    AppConfig cfg = bind(AppConfig.class, props);
    System.out.println("port=" + cfg.port + " name=" + cfg.name
        + " debug=" + cfg.debug);
    System.out.println("实验完成。");
}
}

```

设计说明:任务 2 的 validate 故意"收集错误不抛异常"——表单校验需要一次给用户列全所有问题,抛第一个就返回是差体验;任务 3 的继承链与第 11 章实验同源,但这里每个字段标了"出处类"便于定位;任务 6 的优先级链(环境变量 > 配置 > 默认值)是生产配置系统的标准语义,Spring 的 PropertySource 顺序就是它;正则里的 \s 转义在 Java 字符串里要写 \\s,实验源码已正确处理。扩展方向:校验器支持"注解组"(表单分步校验)、绑定器加类型转换器注册表、用 APT 把校验在编译期生成(性能敏感场景)。

下一章预告:第 13 章 模块系统与构建工具。JPMS 模块化、Maven 依赖管理、jar 的生成与运行——把多个类组织成可交付的制品。